

What is Priority Queue | Introduction to Priority Queue

Last Updated : 12 Mar, 2025

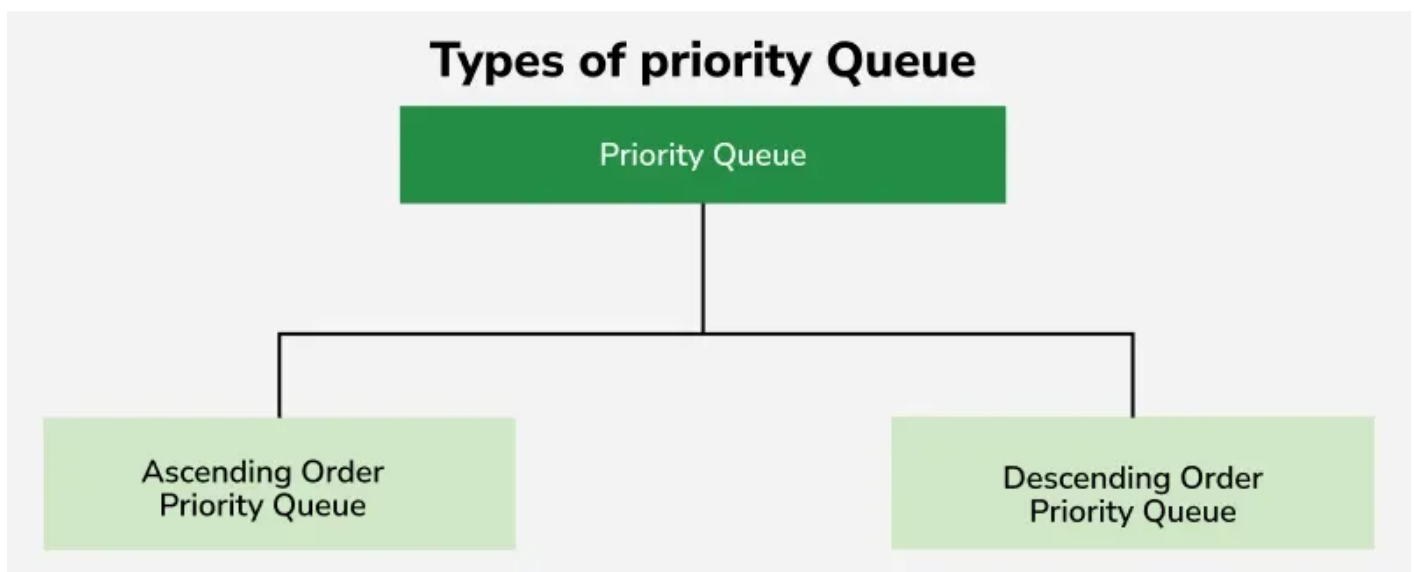


A **priority queue** is a type of queue that arranges elements based on their priority values.

- Each element has a priority associated. When we add an item, it is inserted in a position based on its priority.
- Elements with higher priority are typically retrieved or removed before elements with lower priority.
- [Binary heap](#) is the most common method to implement a priority queue. In binary heaps, we have easy access to the min (in min heap) or max (in max heap) and binary heap being a complete binary tree are easily implemented using arrays. Since we use arrays, we have cache friendliness advantage also.
- Priority Queue is used in algorithms such as [Dijkstra's algorithm](#), [Prim's algorithm](#), and [Huffman Coding](#).

For example, in the below priority queue, an element with a maximum ASCII value will have the highest priority. The elements with higher priority are served first.

Types of Priority Queue



Types of Priority Queue

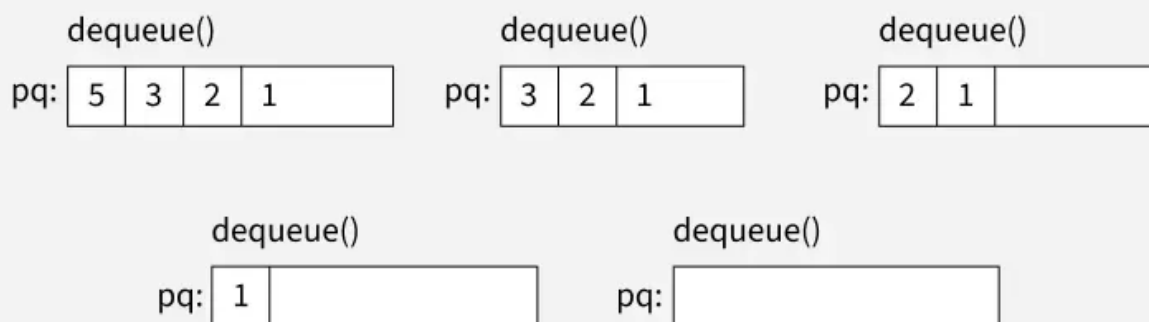
- **Ascending Order Priority Queue** : In this queue, elements with lower values have higher priority. For example, with elements 4, 6, 8, 9, and 10, 4 will be

dequeued first since it has the smallest value, and the dequeue operation will return 4.

- **Descending order Priority Queue** : Elements with higher values have higher priority. The root of the heap is the highest element, and it is dequeued first. The queue adjusts by maintaining the heap property after each insertion or deletion.

The following is an example of Descending order Priority Queue.

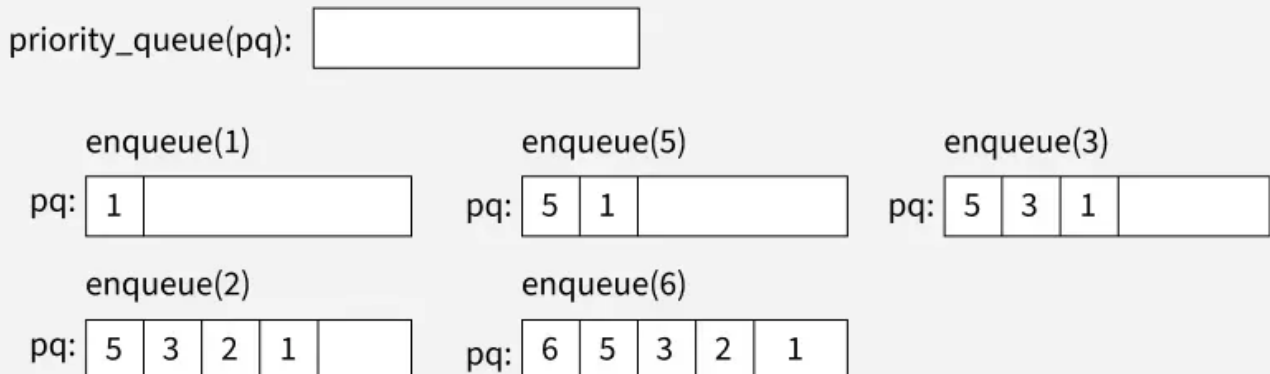
Implementation of Priority Queue



Priority Queue

Implementation of Priority Queue

Input: 1, 5, 3, 2, 6 (Higher number has higher priority)



Priority Queue

How Priority is Determined in a Priority Queue?

In a priority queue, generally, the value of an element is considered for assigning the priority. For example, the element with the highest value is assigned the highest priority and the element with the lowest value is assigned the lowest priority. The **reverse case can also be used i.e.**, the element with the lowest value can be assigned the highest priority. Also, the priority can be assigned according to our needs.

Operations on a Priority Queue

A typical priority queue supports the following operations:

1) Insertion : If the newly inserted item is of the highest priority, then it is inserted at the top. Otherwise, it is inserted in such a way that it is accessible after all higher priority items are accessed.

2) Deletion : We typically remove the highest priority item which is typically available at the top. Once we remove this item, **we need not move next priority item at the top.**

3) Peek : This operation only returns the highest priority item (which is typically available at the top) and does not make any change to the priority queue.

Difference between Priority Queue and Normal Queue

There is no priority attached to elements in a [queue](#), the rule of first-in-first-out(FIFO) is implemented whereas, in a priority queue, the elements have a priority. The elements with higher priority are served first.

Library Implementation of Priority Queue

1. [Priority Queue in C++](#)
2. [Priority Queue in Java](#)
3. [Priority Queue in Python](#)
4. [Priority Queue in JavaScript](#)
5. [Priority Queue in C#](#)

Implementation of Priority Queue

Priority queue can be implemented using the following data structures:

1) Implement Priority Queue Using Array:

A simple implementation is to use an array of the following structure.

```
struct item {  
    int item;  
    int priority;  
}
```

- **enqueue()**: This function is used to insert new data into the queue.
- **dequeue()**: This function removes the element with the highest priority from the queue.
- **peek()/top()**: This function is used to get the highest priority element in the queue without removing it from the queue.

Arrays	enqueue()	dequeue()	peek()
Time Complexity	O(1)	O(n)	O(n)

Please Refer to [Priority Queue using Array](#) for more details.

2) Implement Priority Queue Using Linked List:

In a LinkedList implementation, the entries are sorted in descending order based on their priority. The highest priority element is always added to the front of the priority queue, which is formed using linked lists. The functions like **push()**, **pop()**, and **peek()** are used to implement a priority queue using a linked list and are explained as follows:

- **push()**: This function is used to insert new data into the queue.
- **pop()**: This function removes the element with the highest priority from the queue.
- **peek() / top()**: This function is used to get the highest priority element in the queue without removing it from the queue.

Linked List	push()	pop()	peek()
Time Complexity	O(n)	O(1)	O(1)

Note: We can also use Linked List, time complexity of all operations with linked list remains same as array. The advantage with linked list is deleteHighestPriority() can be more efficient as we don't have to move items.

Please Refer to [Priority Queue using Linked List](#) for more details.

3) Implement Priority Queue Using Heap:

A [Binary Heap](#) is ideal for priority queue implementation as it offers better performance than arrays or linked lists. The largest key is at the top and can be removed in $O(\log n)$ time, with the heap property restored efficiently. If a new entry is inserted immediately, its $O(\log n)$ insertion time may overlap with restoration. Since heaps use contiguous storage, they efficiently handle large datasets while ensuring logarithmic time for insertions and deletions. Operations on Binary Heap are as follows:

- **insert(p):** Inserts a new element with priority p.
- **extractMax():** Extracts an element with maximum priority.
- **remove(i):** Removes an element pointed by an iterator i.
- **getMax():** Returns an element with maximum priority.
- **changePriority(i, p):** Changes the priority of an element pointed by i to p.

Binary Heap	insert()	remove()	peek()
Time Complexity	$O(\log n)$	$O(\log n)$	$O(1)$

Please Refer [Priority Queue using Heap Implementation](#) for more details

4) Implement Priority Queue Using Binary Search Tree:

A Self-Balancing Binary Search Tree like [AVL Tree](#), Red-Black Tree, etc. can also be used to implement a priority queue. Operations like peek(), insert() and delete() can be performed using [BST](#).

Binary Search Tree	peek()	insert()	delete()
Time Complexity	$O(1)$	$O(\log n)$	$O(\log n)$

Please refer [Why is Binary Heap Preferred over BST for Priority Queue?](#) for details.

Problems Based on Priority Queue or Heap

The following list of top coding problems on Heap covers a range of difficulty levels, from easy to hard, to help candidates prepare for interviews.

[Top 50 Problems on Heap Data Structure asked in SDE Interviews](#)

Applications of Priority Queue

- CPU Scheduling
- Graph algorithms like [Dijkstra's shortest path algorithm](#), [Prim's Minimum Spanning Tree](#), etc.
- All [queue applications](#) where priority is involved.
- Data compression in Huffman code
- Event-driven simulation such as customers waiting in a queue.
- Finding Kth largest/smallest element.

Please refer [Applications of Priority Queue](#) for more details.

Advantages of Priority Queue

- Quick access to the highest priority element, as elements are ordered by priority.
- Dynamic reordering when priority values change.
- Improve efficiency in algorithms like Dijkstra's and A* for pathfinding.
- Used in real-time systems for fast retrieval of high-priority elements.

Disadvantages of Priority Queue

- **High complexity:** More difficult to implement and maintain than simpler structures like arrays and linked lists.
- **High memory consumption:** Storing priority values can use more memory, which is a concern in resource-limited systems.
- **Not always the most efficient:** Other structures, like heaps or binary search trees, may be more efficient for certain operations.
- **Less predictable:** Element retrieval order can be less predictable due to dynamic priority-based ordering.

See also:

1. [Recent articles on Priority Queue!](#)

[Comment](#)[More info](#) ▼[Advertise with us](#)[Next Article](#) >[What is Priority Queue | Introduction to Priority Queue](#)

Similar Reads

Why can't a Priority Queue wrap around like an ordinary Queue?

Priority Queue: A priority queue is a special type of queue in which each element is assigned a priority value. And elements are served based on their priority. This means that elements with higher priority ar...

🕒 3 min read

Turn a Queue into a Priority Queue

What is Queue?Queue is an abstract data type that is open at both ends. One end is always used to insert data (enqueue) which is basically the rear/back/tail end and the other which is the front end is use...

🕒 9 min read

Why does Queue have front but Priority-queue has top in stl?

Why does the queue have front but the priority queue has top in stl? The main difference between a queue and a priority queue is that a queue follows the FIFO (First-In-First-Out) principle, while a priority...

🕒 8 min read

Should we declare as Queue or Priority Queue while using Priority Queue in Java?

Queue: Queue is an Interface that extends the collection Interface in Java and this interface belongs to java.util package. A queue is a type of data structure that follows the FIFO (first-in-first-out) order. The...

🕒 3 min read

Efficient way to initialize a priority queue

STL Priority Queue is the implementation of Heap Data Structure. By default, it's a max heap, and can be easily for primitive data types. There are some important applications of it which can be found in this...

🕒 7 min read

Priority Queue using Queue and Heapdict module in Python

Priority Queue is an extension of the queue with the following properties. An element with high priority is dequeued before an element with low priority. If two elements have the same priority, they are served...

🕒 3 min read

Applications of Priority Queue

A Priority Queue is different from a normal queue, because instead of being a "first-in-first-out", values come out in order by priority. It is an abstract data type that captures the idea of a container whose...

🕒 2 min read

Difference between Circular Queue and Priority Queue

Queues are fundamental data structures that are used to store and manage a collection of elements. While both circular queues and priority queues are types of queues, they have distinct characteristics an...

🕒 4 min read

Indexed Priority Queue with Implementation

Priority queue is a data structure in which data is stored on basis of its priority. In an Indexed Priority Queue, data is stored just like standard priority queue and along with this, the value of a data can be...

🕒 8 min read

Implementation of Priority Queue in Javascript

Priority Queue is an extension of Queue having some properties as follows: Each element of the priority queue has a priority associated with it. Elements are added to the queue as per priority. Lowest priority...

🕒 9 min read
